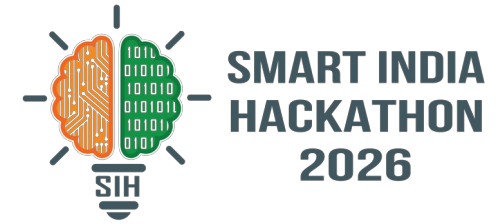


SMART INDIA HACKATHON 2026



TITLE PAGE

Problem Statement ID – SIH26159

Problem Statement Title – SecureMailScope — AI-Assisted Cryptographic Security
Posture Assessment for Secure Email Communications

Organisation – National Technical Research Organisation (NTRO)

Theme – [UNRESOLVED: theme]

PS Category – Software

Team ID – [UNRESOLVED: team id]

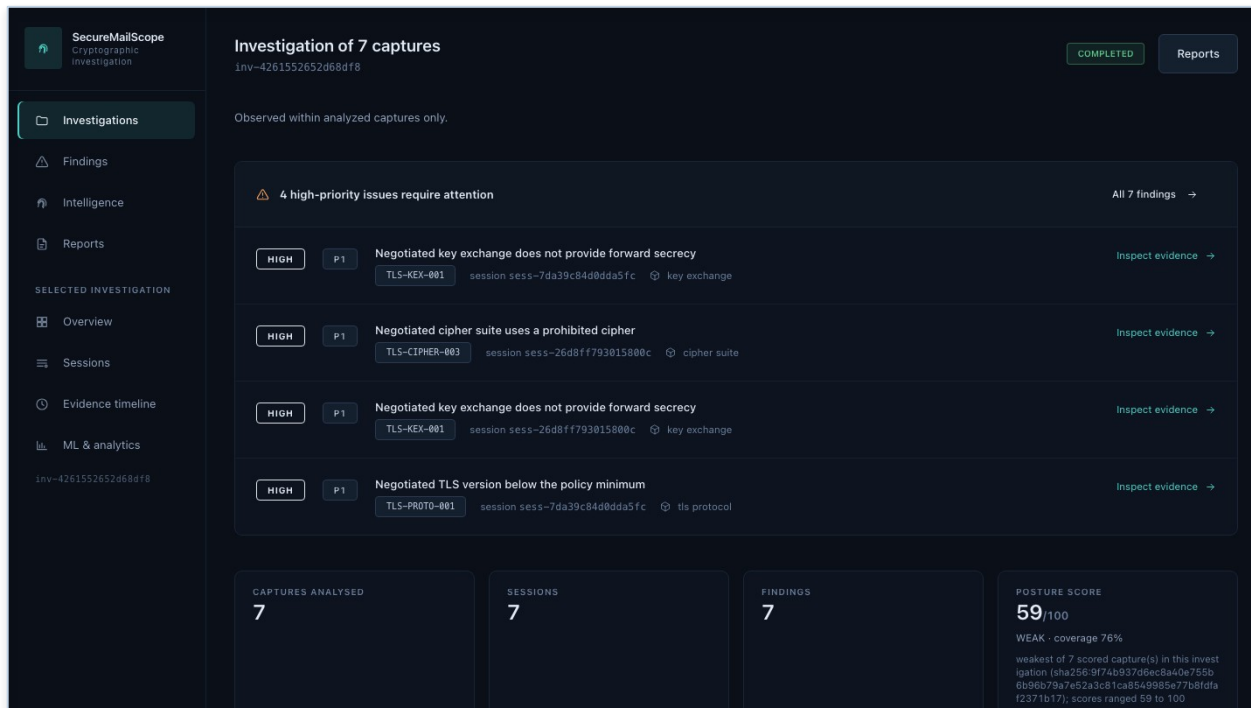
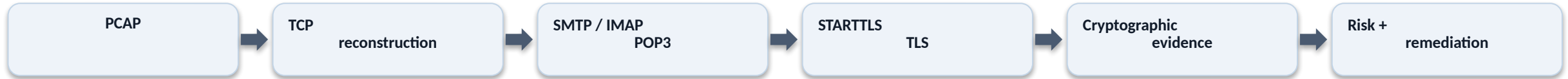
Team Name (Registered on portal) – Zero-Day



IDEA TITLE

❖ SecureMailScope turns passive email packet captures into evidence-backed cryptographic investigations.

Every conclusion is traceable to the packets that establish it — and where a capture cannot show something, it says so.



The investigation overview, on seven synthetic captures. Actual product output.

What it does

Reads authorized PCAP/PCAPNG, reconstructs TCP sessions, identifies the email protocol from the dialogue rather than the port, reconstructs the TLS handshake, extracts certificate evidence where it is observable, applies 25 policy rules and scores the result.

How it addresses SIH26159

The organisation already holds the captures. No probe, no scan, no credential and no change to a production mail server — which also means an authorised investigator with no access to the host can still assess how it protected traffic.

What makes it different

Packet-level provenance on every finding · cryptographic DNA and drift across captures · cross-session correlation · explainable scoring with the arithmetic shown · fully local and offline · stated limits instead of silent gaps

TECHNICAL APPROACH

- Each stage reads only what the previous one produced. Line counts are actual.

PCAP / PCAPNG — untrusted input, 8 hard limits

Safe ingestion — 1,166 loc — format from content, never the extension

TCP reconstruction — ~900 loc — reorder · retransmit · gaps · overlap conflicts

SMTP / IMAP / POP3 — 3,406 loc — real state machines; port is only a hint

STARTTLS / STLS — advertised · requested · outcome · exact upgrade packet

TLS handshake + crypto — 3,042 loc — version from supported_versions, KX from key_share

X.509 intelligence — 861 loc — RFC 5280 path, dates, hostname, key, signature

Security rules + scoring — 3,416 loc — 25 rules · versioned policy · explainable score

Correlation · drift · anomaly — 5,471 loc — fingerprints · entities · blast radius · ML

FastAPI + SQLite — 2,756 loc — loopback only · token · 3 explicit migrations

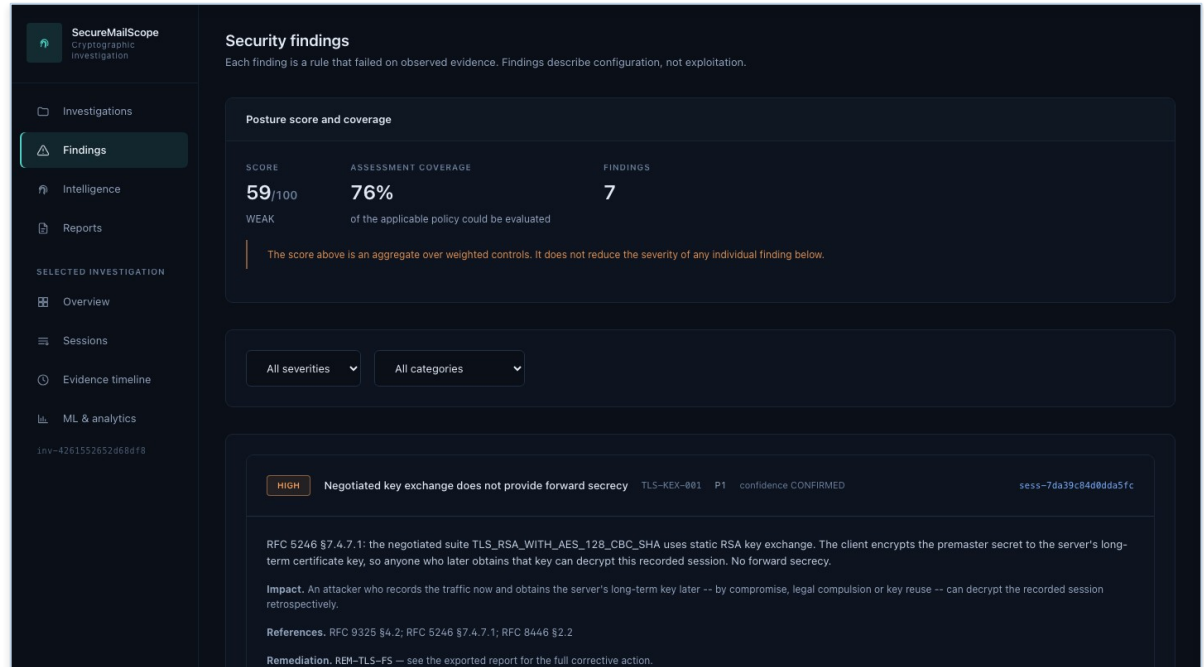
React forensic dashboard — 4,400 loc — TypeScript strict · evidence-linked throughout

JSON · HTML · PDF — 1,582 loc — one canonical model, parity-tested

PASSIVE ONLY

LOCAL FIRST

EVIDENCE LINKED



SecureMailScope
Cryptographic investigation

Security findings
Each finding is a rule that failed on observed evidence. Findings describe configuration, not exploitation.

Posture score and coverage

SCORE	ASSESSMENT COVERAGE	FINDINGS
59/100	76%	7
WEAK of the applicable policy could be evaluated		

The score above is an aggregate over weighted controls. It does not reduce the severity of any individual finding below.

SELECTED INVESTIGATION

- Investigations
- Findings**
- Intelligence
- Reports

Overview

- Sessions
- Evidence timeline
- ML & analytics

Inv-4261552652d68d18

High Negotiated key exchange does not provide forward secrecy TLS-KEX-001 P1 confidence CONFIRMED sess-7da39c84d0da5fc

RFC 5246 §7.4.7.1: the negotiated suite TLS_RSA_WITH_AES_128_CBC_SHA uses static RSA key exchange. The client encrypts the premaster secret to the server's long-term certificate key, so anyone who later obtains that key can decrypt this recorded session. No forward secrecy.

Impact. An attacker who records the traffic now and obtains the server's long-term key later -- by compromise, legal compulsion or key reuse -- can decrypt the recorded session retrospectively.

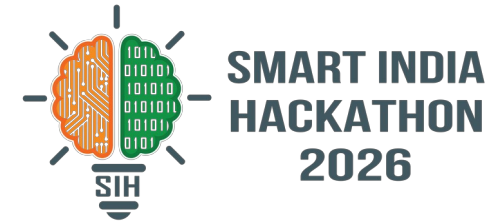
References. RFC 9325 §4.2; RFC 5246 §7.4.7.1; RFC 8446 §2.2

Remediation. REN-TLS-FS — see the exported report for the full corrective action.

Technologies

Python 3.12 · Scapy 2.7 (dissection only) · cryptography 50 · Pydantic 2.9 · FastAPI · SQLite · SQLAlchemy
 React 18 · TypeScript strict · Vite 8 · Tailwind · scikit-learn 1.5 · ReportLab · Jinja2
 Verification: pytest · hypothesis · Playwright · Vitest · TShark used as an independent dissector

FEASIBILITY AND VIABILITY



- Feasible because it is built and measured. Every figure is from the release commit, re-run — not an estimate.

Implemented and verified

Full PCAP → report workflow, end to end
 TCP reconstruction handles reordering, retransmission, gaps, overlap conflicts and tuple reuse
 SMTP / IMAP / POP3 state machines with STARTTLS and STLS; protocol from the dialogue, not the port
 TLS 1.2 and TLS 1.3 handled with separate semantics
 X.509 analysis wherever the evidence permits
 25 deterministic rules, versioned policy, explainable score
 Cross-capture drift and correlation

Verification

1,354 backend tests pass; 1,364 with the TShark cross-check enabled
 Ten cross-checks compare our dissection against Wireshark's, so a bug in our own parser cannot validate itself
 88 frontend tests · 5 browser end-to-end specs against the real backend, nothing mocked
 ruff and mypy clean over 153 files
 Expectations are hand-derived and committed as manifests; the captures are generated, so a test cannot confirm itself
 0 known dependency vulnerabilities (pip-audit, npm audit); hash-pinned lock; CycloneDX SBOM

PROPERTY	STATUS	BEFORE	AFTER	OFFERS COMPARABLE	EXPLANATION
CERTIFICATE_FINGERPRINT	UNCHANGED_WITH_EVIDENCE	875d99 5d18e 3d9da 51fc99 9d1c52 7d093 25d99c 3d0d47 d07144 d2d058 d075 1d0251 87f1c2 d071c8	875d99 5d18e 4d349 6d451 f099a 27c52 7d093 32d59 9c374 9d709 71d4e 2d058 d075 d071c8 7d1771 7d1771	NOT APPLICABLE	Observed as 875d995d919d3d0d47d1c09d27c527d09325d99c3d0d47 in both captures. This is positive evidence of stability, not no change.
CERTIFICATE_PUBLIC_KEY	UNCHANGED_WITH_EVIDENCE	627132 13d8c2 d3d0c2 d07144 d32298	627132 21388 c2d28 4d5ab 7d13e	NOT APPLICABLE	Observed as 62719213d8c2e10d5d47441d322995d0c73d076d4c18d in both captures. This is positive evidence of stability, not no change.

Cryptographic drift across two captures. Actual product output.

Challenges and limits — stated, not hidden

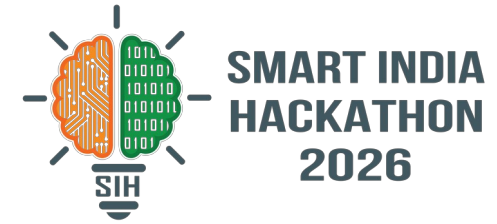
TLS 1.3 encrypts the Certificate message. A passive capture without decryption material cannot expose what is not on the wire. Reported NOT_AVAILABLE with the reason, never blank and never guessed. TLS 1.2 certificates are read and verified in full: dates, chain, hostname, key size, signature algorithm.

The supervised classifier is NOT VALIDATED for real-world deployment. It is trained and measured on controlled synthetic servers (macro-F1 0.5624). Synthetic evaluation does not establish real-world accuracy, the interface says so on screen, and the classifier drives no finding and no score. The anomaly detector actually in use is a deterministic rarity baseline, not a machine-learning model — a trained Isolation Forest was measured, lost, and is shipped labelled as experimental.

Handshake completion is not verifiable and revocation is never checked. A capture carries no traffic keys, and an OSCP or CRL request would break the passive rule. Three constants are asserted false or zero in every report by the test suite.

Deployment. Memory binds before speed — about 76 KB peak resident per packet — and no benchmark has been run on the assumed 4-core/8 GB minimum, which is recorded NOT VERIFIED. Analysis cannot be cancelled, and a test asserts the absence so it cannot be mistaken for a broken control.

IMPACT AND BENEFITS



- For authorised email-security investigation. These are potential benefits: the tool is demonstrated on synthetic captures and has no deployments, no users and no measured real-world outcomes.

DISCOVER

Identify the cryptographic configurations actually negotiated — versions, cipher suites, key exchanges and certificates — from traffic the organisation already holds, without touching the host.

EXPLAIN

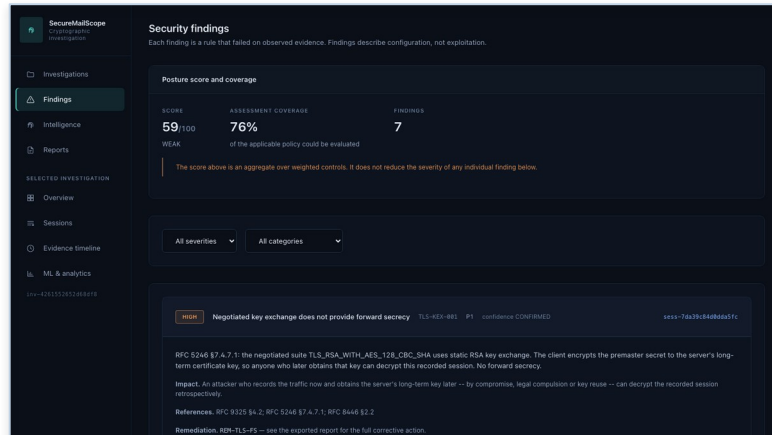
Link every finding to the packets that establish it, with the RFC it applies and the limitation it carries. An analyst can open the same packet in Wireshark and check.

PRIORITIZE

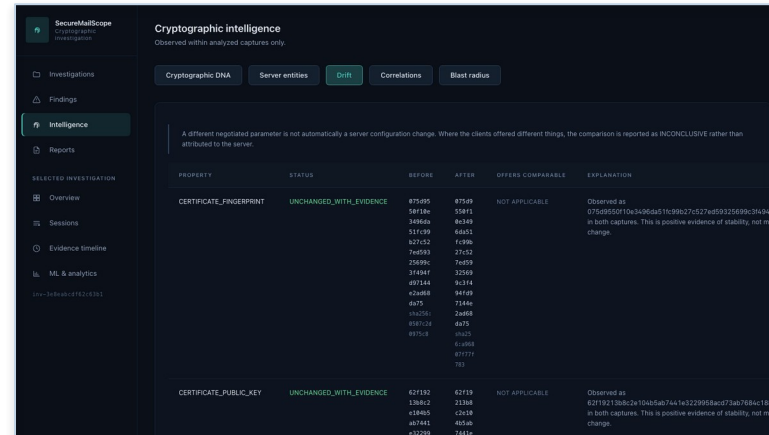
Rank weaknesses by a severity and confidence matrix, each with a named remediation and its expected security effect, so 'what do I fix first' has an answer with a reason.

TRACK

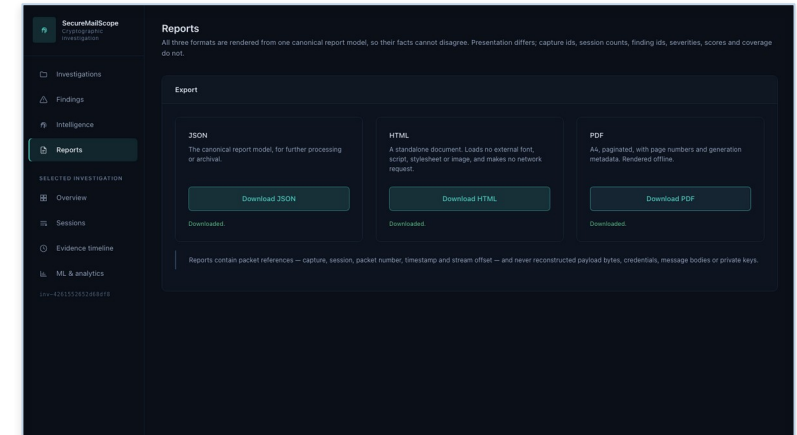
Detect cryptographic drift across captures: when the same observed service changes what it negotiates, and whether the change is attributable to the server or merely to a different client.



A finding, with its rule, severity and remediation.



Configuration drift between two captures.



JSON, offline HTML and PDF export.

Captured enterprise traffic stays local by default — nothing is uploaded, and no host in a capture is ever contacted. • No financial saving, adoption figure, deployment or real-world detection rate is claimed: none has been measured.

- Standards the rules cite and the parsers implement. Every rule names the clause it applies.

TLS and certificates

- [1] RFC 8446 — TLS 1.3
- [2] RFC 5246 — TLS 1.2
- [3] RFC 9325 — Recommendations for secure use of TLS (2022)
- [4] RFC 8996 — Deprecating TLS 1.0 and 1.1
- [5] RFC 7457 — Known attacks on TLS and DTLS
- [6] RFC 4492 — ECC cipher suites for TLS
- [7] RFC 5280 — X.509 certificate and CRL profile
- [8] RFC 6125 — Verifying application service identity
- [9] NIST SP 800-52 Rev. 2 — TLS implementation guidelines
- [10] IANA TLS Parameters registry

Email transport

- [11] RFC 5321 — Simple Mail Transfer Protocol
- [12] RFC 3207 — SMTP over TLS (STARTTLS)
- [13] RFC 9051 — IMAP version 4rev2
- [14] RFC 1939 — Post Office Protocol version 3
- [15] RFC 2595 — Using TLS with IMAP, POP3 and ACAP
- [16] RFC 2606 — Reserved DNS names (.invalid, used by every synthetic fixture)

Libraries

- [17] Scapy · [18] python-cryptography · [19] FastAPI
- [20] scikit-learn · [21] React · [22] Wireshark / TShark

Verification methodology

Deterministic synthetic fixtures. Every capture is generated from a fixed seed, so the same bytes are produced on every machine and in CI.

Known-answer manifests. Expectations are hand-derived and committed; the captures themselves are gitignored. A test cannot confirm its own output.

Independent TShark cross-checks. Ten comparisons against Wireshark's dissector, so a shared bug in our parser cannot validate itself.

Browser end-to-end tests. Five Playwright specs drive the real backend over the real engine, including a restart to prove persistence.

Report-parity verification. One canonical model feeds JSON, HTML and PDF, and their facts are asserted equal.

Project research, written alongside the code

docs/evidence-model.md — the four evidence statuses and how provenance is tracked · docs/scoring-methodology.md — the formula, the bands, and why a multi-capture headline is the weakest capture rather than an average

docs/ml-methodology.md, ml-evaluation.md, ml-model-card.md — why a deterministic baseline was selected over a trained model by measurement, and what neither is fit for · docs/limitations.md — what passive analysis cannot do

docs/security-audit.md — threat model, controls, and what the audit did not cover · docs/performance-benchmarks.md — thresholds committed before measurement, including a published failing run · submission/final/REQUIREMENT-COVERAGE.md — all 15 problem-statement requirements with status and evidence